Engenharia de Contexto

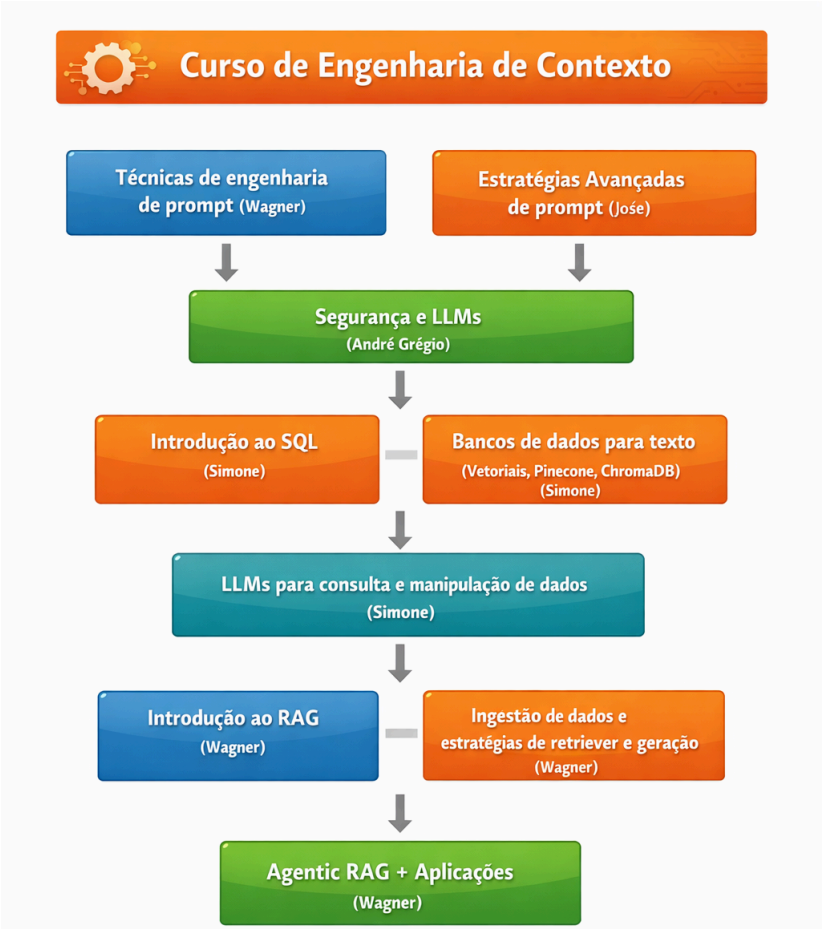
Especialização em Inteligência Artificial Generativa

Prof. Dr. Wagner Hugo Bonat

Estrutura do curso

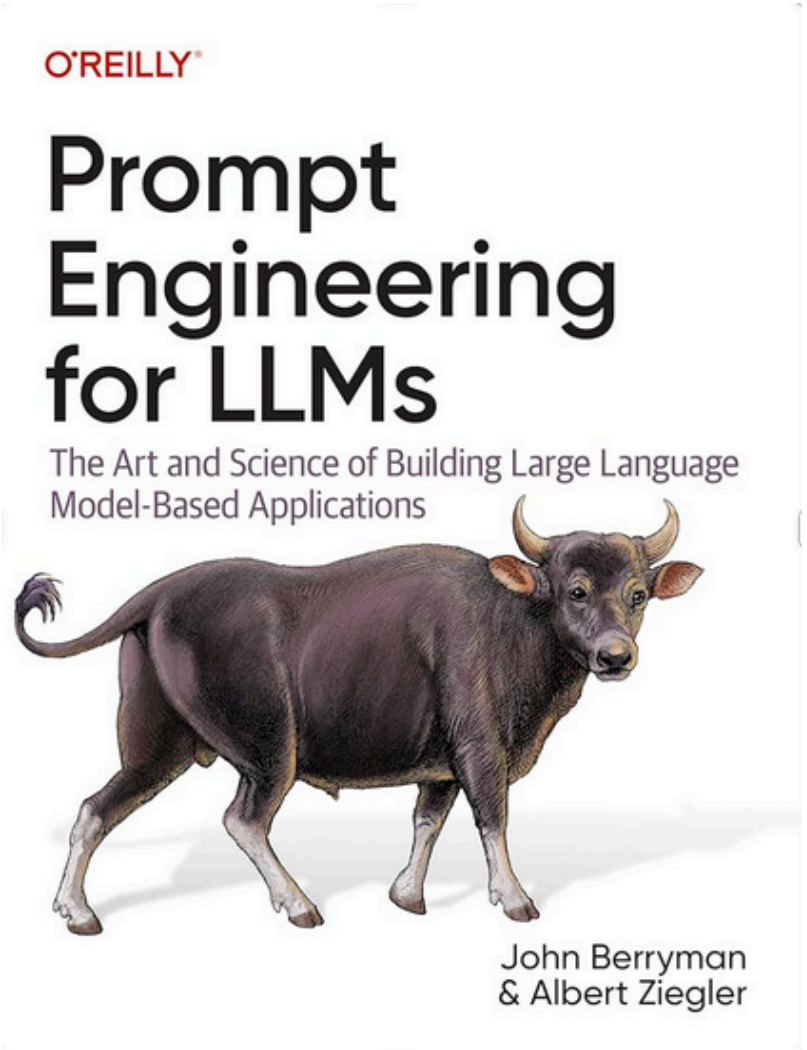
O que você vai aprender neste curso?

- **Técnicas de engenharia de prompt.** (Prof. Wagner)
- **Estratégias Avançadas de prompt.** (Prof. José)
- **Segurança e LLMs.** (Prof. André Grégio)
- Introdução ao SQL. (Prof. Simone)
- Bancos de dados para texto. (Profa. Simone)
- LLMs para consulta e manipulação de dados. (Profa. Simone)
- Introdução ao RAG. (Profa. Wagner)
- Ingestão, *retriever* e geração. (Prof. Wagner)
- Agentic RAG + Aplicações (Prof. Wagner)



Referência bibliográfica

- Área em constante movimento.
- Novidade todo dia!
- Vamos focar principalmente nos fundamentos.
- Ferramenta agnóstico - apesar de usar exemplos em `chatatlas` e `langchain`.
- Documentação das bibliotecas tem muito conhecimento!
- Autores do livro foram da equipe do Github Copilot.
- Livro de 2024 - já está defasado :(
- Materiais complementares no moodle.
- Engenharia de Contexto
- Anthropic Academy



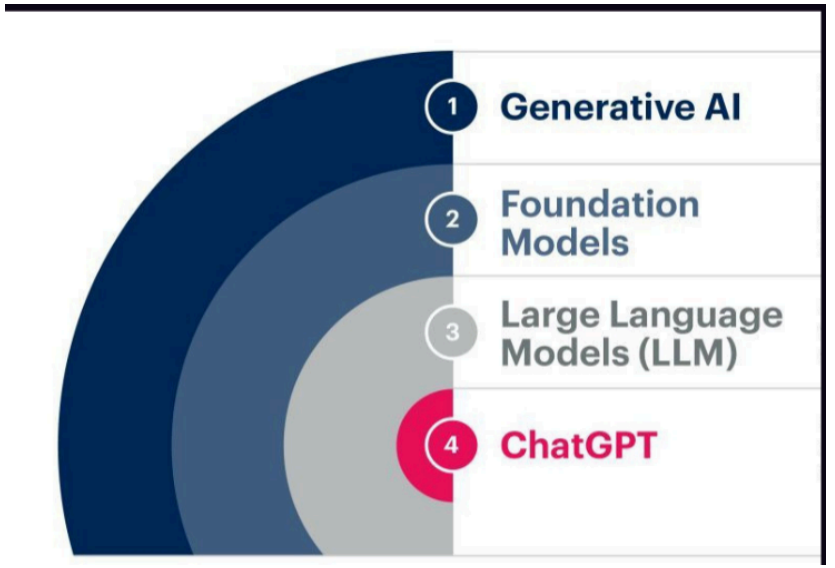
Breve contexto e introdução

O que são LLMs?

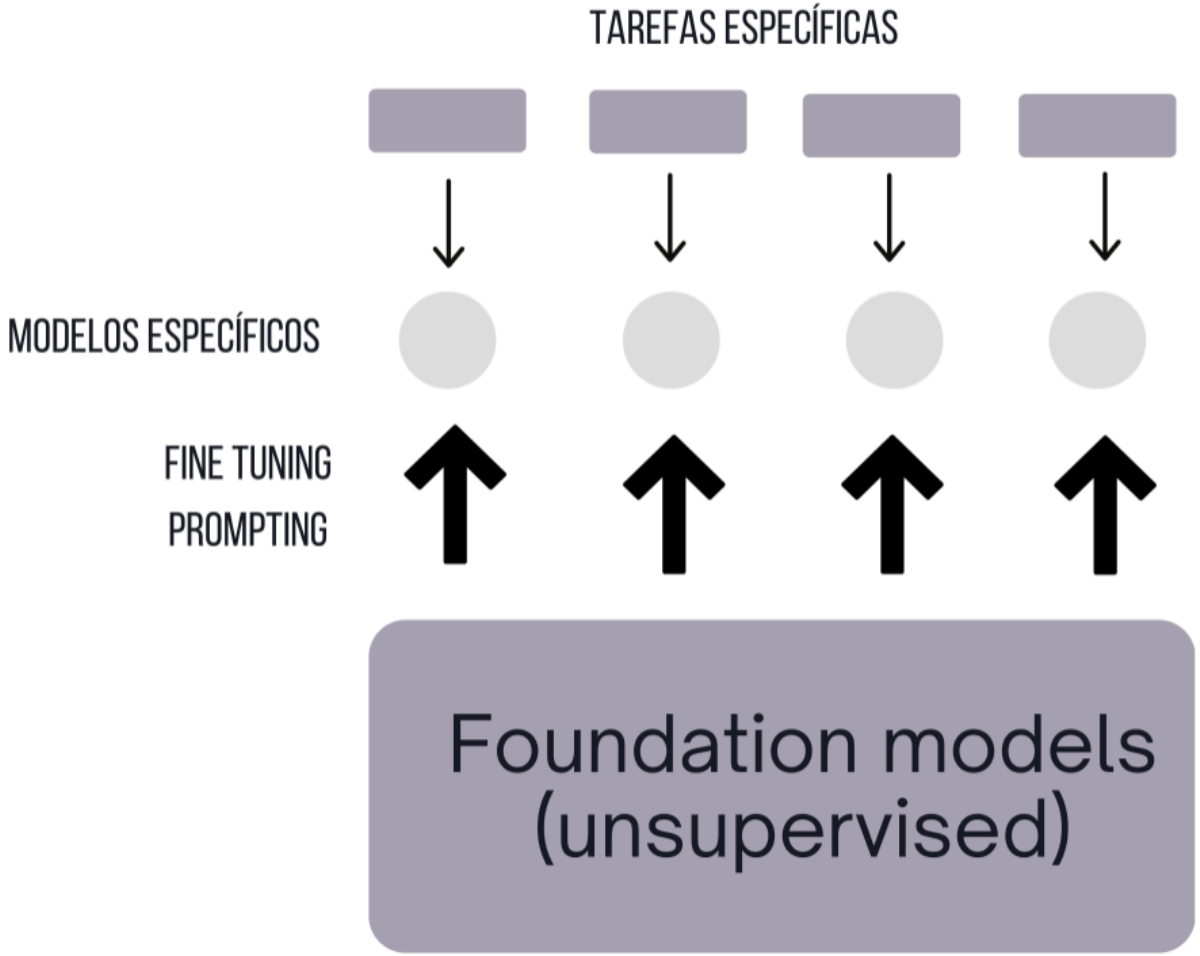
- LLMs são modelos que predizem a próxima palavra em um bloco de palavras!
- Palavras da própria OpenAI

> Our model, called GPT-2 (a sucessor to GPT), was trained simply to predict the next word in 40 GB of Internet text. Due to our concerns about malicious applications of the technology, we are not releasing the trained model. (OpenAI blog post, 2019).

- GPT - *Generative pre-trained transformer architecture*.
- Por que tanto medo da tecnologia?
  - O modelo se tornou muito bom em imitar a linguagem.
- A evolução é realmente fantástica.
- Rodar o `Script1.py`.



Quebra de paradigma



VANTAGENS

- PERFORMANCE
- PRODUTIVIDADE

DESVANTAGENS

- COMPUTAÇÃO
- CONFIANÇA

APLICAÇÕES

- TEXTO
- CÓDIGO
- IMAGENS
- VÍDEOS

Breve histórico

1950s–1990s
Initial attempts are made to map hard rules around languages and follow logical steps to accomplish tasks like translating a sentence from one language to another.

While this works sometimes, strictly defined rules only work for concrete, well-defined tasks that the system has knowledge about.

1990s
Language models begin evolving into statistical models and language patterns start being analyzed, but larger-scale projects are limited by computing power.

2000s
Advancements in machine learning increase the complexity of language models, and the wide adoption of the internet sees an enormous increase in available training data.

2012
Advancements in deep learning architectures and larger data sets lead to the development of GPT (Generative Pre-trained Transformer).

2018
Google introduces BERT (Bidirectional Encoder Representations from Transformers), which is a big leap in architecture and paves the way for future large language models.

2020
OpenAI releases GPT-3, which becomes the largest model at 175B parameters and sets a new performance benchmark for language-related tasks.

2022
ChatGPT is launched, which turns GPT-3 and similar models into a service that is widely accessible to users through a web interface and kicks off a huge increase in public awareness of LLMs and generative AI.

2023
Open source LLMs begin showing increasingly impressive results with releases such as Dolly 2.0, LLaMA, Alpaca and Vicuna.
GPT-4 is also released, setting a new benchmark for both parameter size and performance.

- Foundation Large Language Models and Text Generation.

Detalhes sobre a família GPT de modelos.

Model	Release Date	Parameter Count	Training Data	Training Cost
GPT-1	June 11, 2018	117 million	BookCorpus: 4.5 GB of text from 7,000 unpublished books	Not disclosed
GPT-2	February 14, 2019 (initial); November 5, 2019 (full)	1.5 billion	WebText: 40 GB of text, 8M documents from 45M Reddit-upvoted pages	Not disclosed
GPT-3	May 28, 2020	175 billion	499B tokens: Common Crawl (570 GB), WebText, Wikipedia, Books1 & Books2	1.5e21 FLOP
GPT-3.5	March 15, 2022	175 billion	Undisclosed	3.1e23 FLOP
GPT-4	March 14, 2023	1.8 trillion (rumored)	Rumored 13T tokens	~2.1e25 FLOP (est.)

- A notação 1.5e21 FLOP é uma forma científica de escrever números muito grandes.
- Isso equivale a: 1.500.000.000.000.000.000.000 operações de ponto flutuante.

Engenharia de Contexto

O que é Engenharia de Contexto?

- A prática de projetar, estruturar e controlar o **fluxo de informações** fornecidas a um LLM.

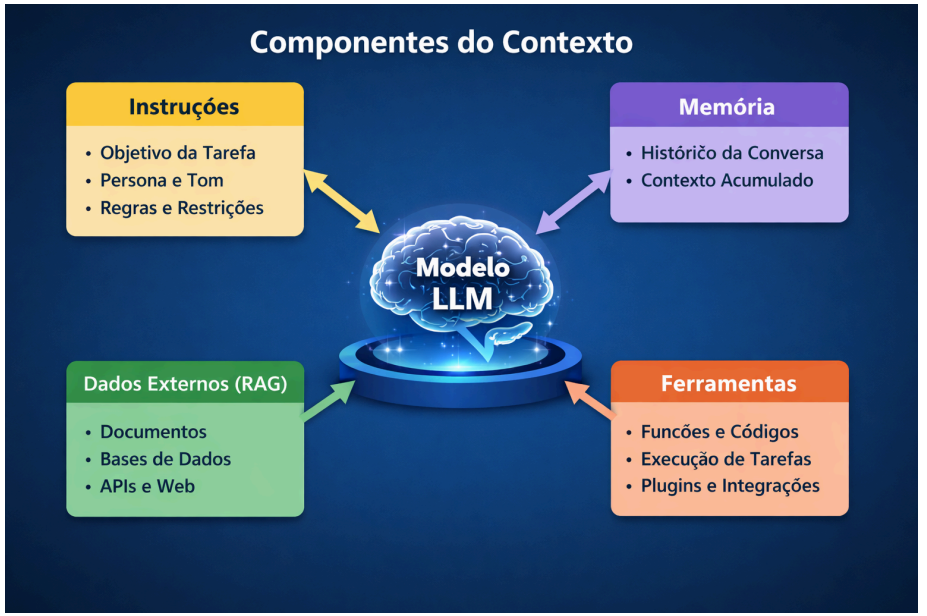
Papel no sistema

- Define **o que o modelo sabe naquele momento**.
- Controla **qualidade, coerência e custo** das respostas.

Qual a importância?

- Define diretamente a qualidade da resposta.
- Controla alucinações e ambiguidade.
- Comportamento reproduzível e confiável.

Componentes do contexto



- **Lembre-se**: O LLM responde com base no contexto disponível!
- Pode ser o que ele teve no treinamento, o que você está fornecendo ou ambos.

Engenharia de Prompt

O que é Engenharia de Prompt?

- Estratégias para projetar **documentos de entrada (prompts)** que orientam como o LLM deve processar o contexto e gerar respostas.

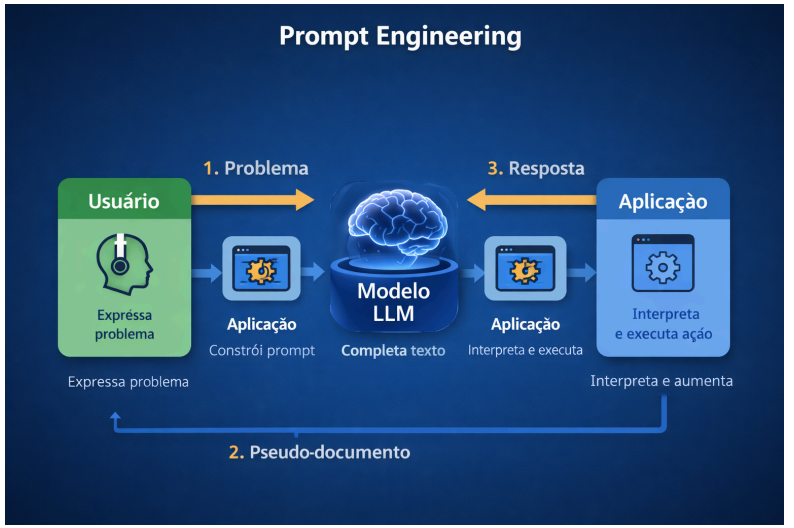
Papel no sistema

- Define como o modelo deve pensar e responder dado o contexto.

Qual a importância?

- Traduzir problemas do mundo real → formato textual.
- Induzir comportamentos desejados.
- Maximizar a qualidade e utilidade da saída.

Fluxo de informações



- **Contexto** → o que o modelo tem acesso para a resposta.
- **Prompt** → como o modelo age sobre o que tem acesso.

Contexto + Prompt

Como se conectam?

- Usuário → Aplicação → **[Contexto + Prompt]** → LLM → Resposta.
- O contexto fornece a **matéria-prima**.
- O prompt define a **forma de processamento**.
- O LLM executa a **completação do texto**.

Qualidade da saída = f(Contexto, Prompt)

Princípios de Engenharia

1. Garbage in, garbage out.
   - Contexto ruim → resposta ruim.
2. Instruções importam.
   - Mesmo contexto, prompts diferentes → resultados diferentes.
3. Trade-off fundamental.
   - Mais contexto ≠ melhor.
   - Relevância > quantidade.
4. Separação de responsabilidades.
   - Contexto → dados.
   - Prompt → controle.

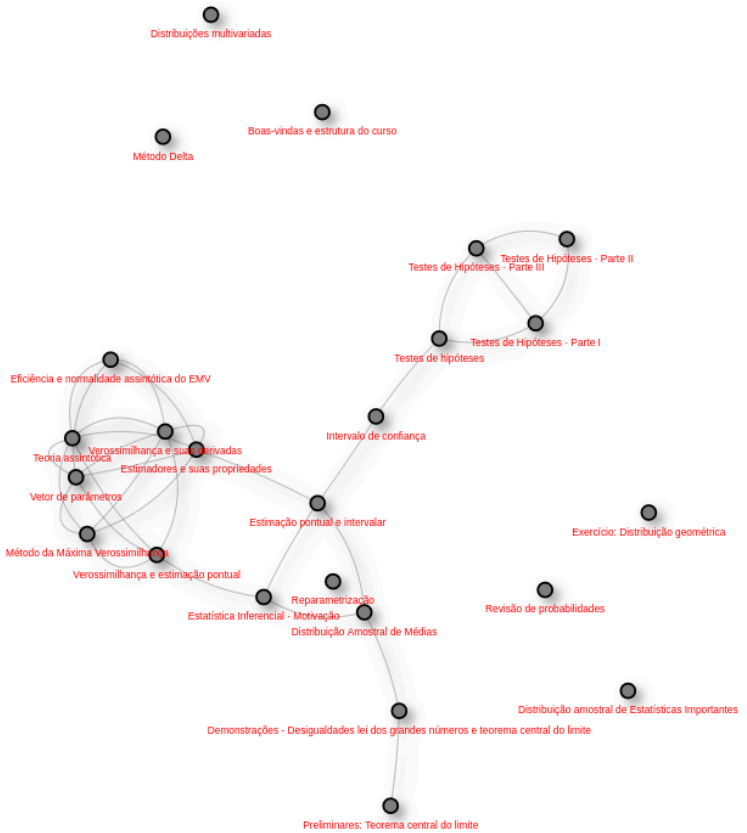
Exemplo conceitual

Assistente virtual: Curso de Inferência Estatística

Visão geral

- Assistente baseado em **RAG (Retrieval-Augmented Generation)**.
- Baseado em **25 vídeos** de aulas de inferência estatística.
- **Comportamento esperado**:
  - Responder apenas perguntas sobre inferência estatística baseado no conteúdo das minhas aulas.
  - Indicar link para os vídeos do *youtube* para o aluno ter mais detalhes.
- **Link acesso**: http://200.17.213.118:8503/ -> Use email: wbonat@ufpr.br

Estrutura de rede entre os vídeos



Assistente virtual: Curso de Inferência Estatística

Pipeline de dados

1. Transcrição dos vídeos.
2. *Chunking* (segmentos de ~5 minutos).
3. *Embeddings*.
4. Armazenamento em **ChromaDB (vector store)**.

Principais componentes

- **Retriever**: busca semântica nos chunks.
- **LLM (ChatOpenAI)**: geração da resposta.
- **Prompt**: controla comportamento e formato.
- **Memória**: histórico da conversa.
- **Classificador**: filtra perguntas fora do escopo.

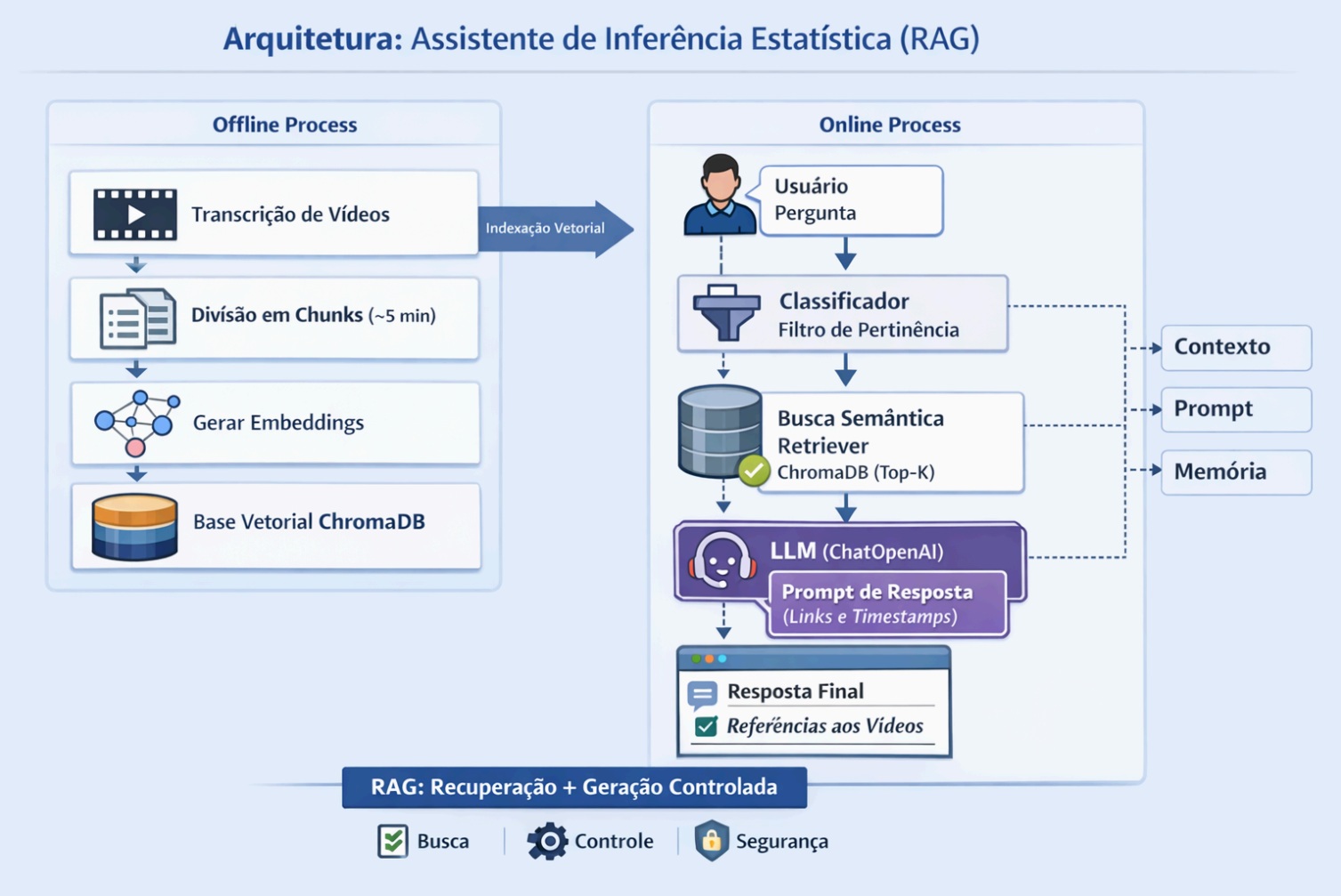
Engenharia de contexto

- Instruções claras:
  - Responder apenas com base no contexto.
  - Incluir **links + timestamps**.
  - Responder em português.
- Reformulação de perguntas (*history aware*).

Mecanismo de controle

- **Filtro de contexto:** Evita perguntas fora do escopo.
- **Grounding (RAG):** Reduz a possibilidade de alucinação.
- **Restrição de output:** Sem código apenas baseado nos documentos.

Arquitetura básica: Assistente de Inferência Estatística



Entendendo LLMs

Large Language Models - LLMs

O que é um LLM?

- Modelo baseado em **redes neurais do tipo Transformer**.
- Treinado para **prever o próximo token** em uma sequência de texto.
- LLM é um serviço que recebe uma *string* de entrada e retorna uma *string* de saída.

O que é um *token*?

- Uma estratégia de converter palavras para números.
- *Tokenizer*: Conversor de palavras para números.
- Exemplo de tokenização.

O que é um embedding?

- Representação vetorial de dados (ex: texto) em um espaço numérico.
- Textos com significados semelhantes → vetores próximos.
- Textos diferentes → vetores distantes.

Como funciona?

1. Texto → **Tokenização**.
2. Tokens → **Embeddings**.
3. Processamento via **Self-Attention (Transformer)**.
4. Saída → **Distribuição de probabilidade**.
5. Geração → **Amostragem de tokens**.

Prompt e conversa

O que é um *prompt*?

- Temos pelo menos três tipos de *prompts*:
- **Core system prompt**: Não é alterável. Setado diretamente pelo fornecedor.
- Exemplo da Anthropic.
- **System prompt**: É setado quando começamos uma conversação vai afetar todas as respostas. É uma espécie de 'estilo' que você declara para o seu assistente.
- **User prompt**: Basicamente é a pergunta que o usuário encaminha para o modelo.

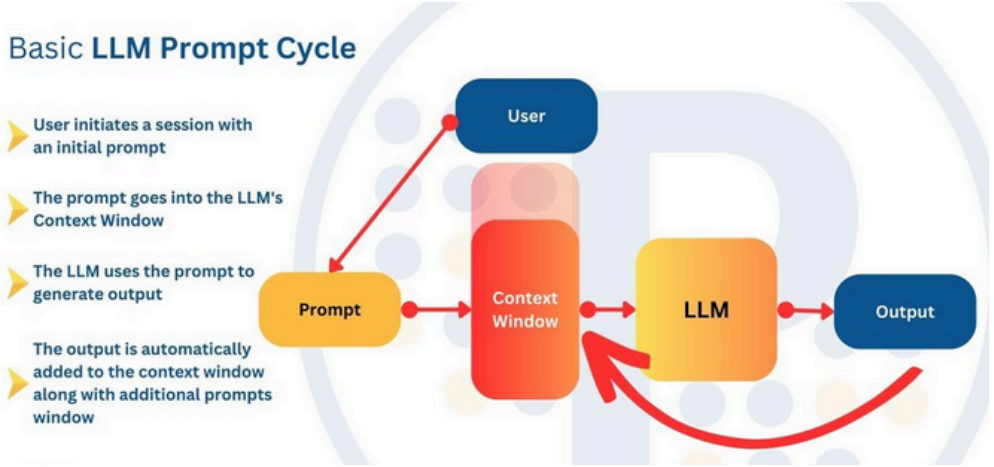
O que é uma conversação?

- Uma série de troca de HTTP *requests* e *responses*.

Fluxo de informações

- Tudo começa com um *prompt*: Tipicamente uma pergunta ou um pedido enviado ao LLM.
- O *prompt* começa uma **conversa**: Uma sequência de trocas entre o usuário e o LLM.
- **Janela de contexto**: Todas as trocas com o LLM.
- Dentro do modelo tanto o *prompt* como a **resposta** do modelo são *tokens*.
- *Tokens* são usados para calcular o custo do uso do modelo e também o tamanho do **contexto**.
- Distinção entre **fornecedores** e **modelos**.
- *Prompt design* é super importante!

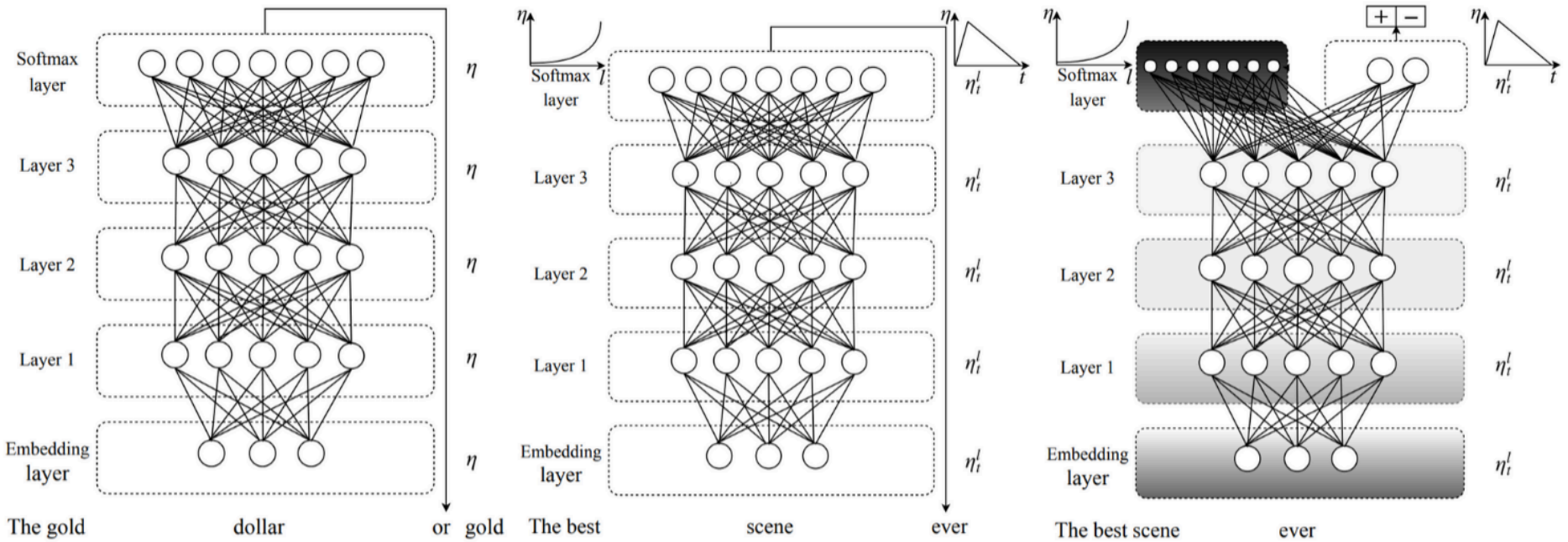
Fluxo



Créditos: https://promptengineering.org

O treinamento e suas implicações

Etapas do treinamento



https://paperswithcode.com/method/ulmfit

O que cada etapa faz?

LM PRE-TRAINING

Nesta etapa uma rede neural profunda é treinada apenas para predizer o próximo token baseado em uma sequência de tokens. Neste passo em geral o modelo não gera textos tão fluídos.

LM FINE-TUNING

Nesta etapa o modelo é treinado para exercer tarefas específicas, como escrever código em determinada linguagem. Conjuntos de dados específicos com perguntas e respostas criadas por humanos são utilizados.

CLASSIFIER FINE-TUNING

Nesta etapa é onde entra a técnica chamada de RLHF (Reinforment Learning from Human Feedback). A ideia é que o modelo é treinado com o suporte de humanos para responder a questões e escrever textos conforme um humano o faria.

O Poder dos Large Language Models (LLMs)

O que LLMs realmente fazem?

- LLMs preveem a próxima palavra em uma sequência.
- Permite uma gama surpreendente de capacidades.
- Não é "raciocínio" ou "pensamento" no sentido humano.

Por que eles funcionam tão bem?

- **Transformers:** Aprendem a misturar contextos de palavras de forma eficaz para prever a próxima palavra.
- **Treinamento massivo:** Alimentados com vastas quantidades de dados da internet, incluindo livros, blogs, artigos e discussões.

A diversidade da Internet

- LLMs foram expostos a bilhões de exemplos de quase todos os tópicos imagináveis.
  - Conversas de todos os tipos.
  - Poemas e letras de músicas.
  - Trabalhos de casa e suas soluções.
  - Perguntas e respostas de testes padronizados.
  - Planos de férias.
  - Exemplos de código e correções (como em Stack Overflow).
  - Conhecimentos de senso comum e fatos científicos.
  - Técnicas de resumo, reescrita e persuasão.

Os modelos são inteligentes?

A Explicação Simples do "Gênio" dos LLMs

- Quando um LLM faz algo "inteligente", é porque ele viu bilhões de exemplos similares.
- Combinam informações já processada para criar respostas razoáveis.
- Isso é inteligente? Repetição de muitos exemplos.

Evite a antropomorfização

- Não precisamos atribuir inteligência, criatividade ou compreensão humana aos LLMs.
- Eles são ferramentas poderosas que operam com base em padrões estatísticos e grandes volumes de dados.

A resposta "padrão"

- Devido à forma como funcionam e são treinados, os LLMs tendem a fornecer respostas "medianas" ou "médias".
- Isso significa que o resultado é o que muitas pessoas (escrevendo na internet) poderiam produzir se tivessem que "chegar a um compromisso".
- A resposta pode ser boa, mas geralmente não é excepcionalmente original ou inovadora.

Vai usar? Então, fique esperto!

O problema do Treinamento

- LLMs são treinados com dados da internet.
- Podem reproduzir linguagem racista, sexista, insultos e desinformação.
- O modelo reflete a diversidade, nem sempre positiva, dos dados de treinamento.

Ausência de "Crenças"

- LLMs não possuem crenças ou opiniões próprias.
- Eles são "adivinhadores de palavras".
- Podem gerar conteúdo a favor ou contra um tema, sem qualquer convicção.

Sem senso de verdade ou moral

- LLMs não distinguem verdade de mentira.
- Podem afirmar que a Terra é redonda ou plana, pois ambos os conceitos existem na internet.
- Refletem o que a internet (base de treino) tem com maior frequência.

Erros e alucinações

- LLMs podem cometer erros devido a inconsistências nos dados ou falhas na atenção.
- Podem "alucinar", gerando palavras não derivadas da entrada ou incorretas.
- Podem inventar nomes de autores ou preferir nomes mais comuns.

Vai usar? Então, fique esperto!

A importância do Contexto/Prompt

- A qualidade da resposta depende do contexto/prompt de entrada.
- Quanto mais informações e especificidade você fornecer, mais especializada e precisa será a resposta.
- Experimente diferentes prompts para obter os melhores resultados.

"Conversas" e Memória

- Você não está realmente "conversando" com um LLM; ele não "lembra" da interação.
- A ilusão de memória é um "truque de programação".
- Há um limite de palavras que podem ser processadas (janela de contexto).

Sem resolução de problemas

- LLMs não têm objetivos ou fazem "look-ahead" como humanos ao planejar.
- Eles são "backward-looking" (voltados para trás).
- O LLM não "pensou nas alternativas" ou considerou cenários futuros.

Sempre verifique as saídas

- É fundamental verificar as respostas de um LLM, especialmente em tarefas de alto risco.
- Para tarefas de baixo risco (ex: contos), erros podem ser aceitáveis.
- Para decisões financeiras (ex: investimentos), erros podem ser muito custosos.

Quais as principais aplicações?



Vantagens, desafios e limitações

Vantagens

- **Eficiência e acurácia**

Performance superior a modelos treinados para tarefas especificas. Ex. análise de sentimento, tradutor, chatbots.

- **Flexibilidade**

Versátil para lidar com diferentes tarefas sem precisar de treinamento adicional e/ou específico.

- **Escalabilidade e Produtividade**

Habilidade para processar grandes quantidades de dados e produzir respostas coerentes.

Desafios e limitações

- **Viés E fairness**

Nem toda o dado encontrado é de boa qualidade. Como saber se o modelo está “dizendo a verdade”?

- **Ética**

Como controlar a privacidade? Uso de conteúdo com direitos autorais.

- **Interpretabilidade**

Desafio é entender e explicar porque o LLM escreve o que escreveu!

Usando LLMs

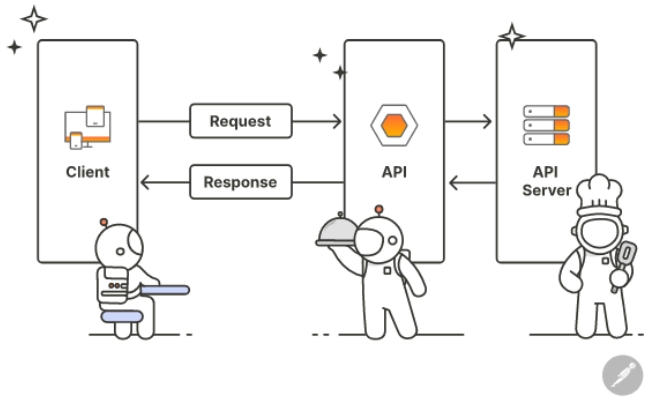
O que são APIs?

- **API (Application Programming Interface)** é um conjunto de rotinas que permitem diferentes softwares interagirem.
- Pense como uma espécie de **ponte** ou **garçom** entre sistemas/softwares.
- Por meio de uma API você pode usar modelos de linguagem como o GPT-4o programaticamente.
- Uma API é o que vai permitir criar novas soluções usando modelos existentes.

Como funciona?

- **Requisição**: Você envia um 'pedido' para a API.
- **Processamento**: O LLM processa o pedido e gera uma resposta.
- **Resposta**: A API devolve a resposta para você.

Fluxo de chamada de API.



https://www.postman.com/what-is-an-api/

Por que usar APIs?

- **Facilidade de integração**: Conecta facilmente modelos de IA com aplicativos.
- **Escalabilidade**: Permite processar grandes volumes de dados sem a necessidade de infraestrutura própria.
- **Atualizações constantes**: Modelos são atualizados com frequência. Manter tudo atualizado "sozinho" é uma tarefa bem complicada.

O que você vai precisar?

- Escolher o seu fornecer de API.
- Obter uma chave de API válida.
- Escolher qual modelo utilizar.
- Ambiente de desenvolvimento configurado.
- Ler a documentação da API ;)
- **Script 1 · Exemplo de chamada de API**.

Exemplo em `python`

```python
import os
import requests
from dotenv import load_dotenv

# Carrega as variáveis de ambiente do arquivo .env
load_dotenv()

# Obtém a chave da API do ambiente
api_key = os.getenv('OPENAI_API_KEY')

# Define o endpoint da API
url = "https://api.openai.com/v1/chat/completions"

# Define os headers da requisição
headers = {
    "Content-Type": "application/json",
    "Authorization": f"Bearer {api_key}"
}

# Define o corpo da requisição
data = {
    "model": "gpt-3.5-turbo",
    "messages": [{"role": "user", "content": "Olá, como você está?"}],
    "max_tokens": 50
}

# Faz a requisição POST para a API
response = requests.post(url, headers=headers, json=data)
```

Sketch chamada API usando python.

Hiperparâmetros em LLMs

O que são?

- Parâmetros definidos **antes da execução do modelo**.
- Controlam **como o modelo gera respostas**, sem alterar seus pesos.
- Diferente dos parâmetros do modelo (pesos), os hiperparâmetros controlam o **comportamento da inferência**.

Principais funções

- Controle de saída → max_tokens e stop.
- Controle de comportamento → temperature e logit_bias.
- Observabilidade → logprobs e top_logprobs.
- Exploração → n.
- UX → stream.

Principais parâmetros da API de Chat Completion

Hiperparâmetro	Função	Impacto no comportamento	Uso típico
temperature	Controla aleatoriedade da geração	Baixo → determinístico; Alto → criativo	0–0.3 (QA), 0.7+ (criativo)
top_p	Amostragem por núcleo (nucleus sampling)	Limita tokens ao conjunto mais provável	0.8–1.0 (equilíbrio)
max_tokens	Limita o tamanho da resposta	Controla custo e extensão	Ajustar conforme tarefa
stop	Define sequências de parada	Interrompe geração em padrões específicos	JSON, listas, outputs estruturados
n	Número de respostas geradas	Permite explorar múltiplas alternativas	Avaliação offline
logit_bias	Ajusta probabilidade de tokens específicos	Força ou evita palavras/tokens	Casos específicos
seed	Controla reprodutibilidade	Mesma entrada → mesma saída (quando suportado)	Testes e debugging
stream	Retorna tokens em tempo real		

Regras do 'dedão'

Caso de Uso	Temperatura	Top_p	Descrição
Geração de Código	0.2	0.1	Gera código que segue padrões e convenções estabelecidos. A saída é determinística e focada. Útil para gerar código sintaticamente correto.
Escrita Criativa	0.7	0.8	Gera textos criativos e diversos para narrativas. A saída é exploratória e menos limitada por padrões.
Respostas de Chatbot	0.5	0.5	Gera respostas conversacionais que equilibram coerência e diversidade. A saída é mais natural e envolvente.
Geração de Comentários de Código	0.3	0.2	Gera comentários de código concisos e relevantes. A saída é determinística e segue convenções.
Scripts de Análise de Dados	0.2	0.1	Gera scripts de análise de dados corretos e eficientes. A saída é determinística e focada.
Escrita de Código Exploratória	0.6	0.7	Gera código que explora soluções alternativas e abordagens criativas. A saída é menos limitada por padrões estabelecidos.

- Na prática teste o quanto for possível para a sua aplicadação.

Principais fornecedores e modelos

- Anthropic (Claude).
- Google Gemini.
- OpenAI.
- PerplexityAI.
- AWS Bedrock.
- Azure chat.
- Github Marktplace.
- Groq.

Open source

- Ollama.
- Hugging Face.
- Llama.
- DeepSeek.
- Mistral AI

Resumo e principais objetivos

- Multimodalidade: Google Gemini, DeepSeek, OpenAI.
- Código e Programação: GitHub Copilot, OpenAI Codex, Mistral AI, Claude Code.
- IA Corporativa: Azure, AWS Bedrock, Anthropic Claude.
- Modelos Open-source: Hugging Face, LLaMA, Mistral AI, DeepSeek, Ollama.
- Performance e Latência: Groq, Mistral AI.
- **Todo dia muda alguma coisa!**

Diretrizes para escolha de fornecedores/modelos

- Em geral em empresas de grande porte você terá acesso a algum *big cloud provider* tais como: Azure, AWS bedrock, databricks ou snowflake.
- OpenAI e Anthropic são ótimos para iniciar em seu próprio computador.
  - Não tem necessidade de uma infraestrutura computacional avançada.
  - Modelos tradicionais: `GPT-4o`, `GPT-3.5-turbo`, `gpt-4o-mini`, `Claude 3.5 Sonnet`.
  - Paga apenas o que usar. Sistema de incluir créditos.
- Ollama: Permite rodar modelos *open source* localmente.
  - Precisa de bons recursos computacionais. Evetualmente GPU para modelos maiores.
- Hugging Face: Repositório com inúmeros modelos *open source* muitos customizados para tarefas específicas.

Fator custo (tome cuidado)!

- Custo: O custo depende do fornecedor e do modelo utilizado. Em geral é cobrado um valor por milhão de **tokens**.
- Exemplos de custo por modelos:
  - Tabela de preço OpenAI.
  - Tabela de preço Vertex AI.
  - Tabela de preço Anthopic.
- Em geral você vai precisar de um cartão de crédito.
- Sistema de crédito (primeiro paga depois usa :)
- Só paga o que for usando.
- Exemplo da minha conta na OpenAI.

Outros fatores que influenciam a escolha do fornecedor/modelo

Fator	Fornecedores Relevantes
Modelos e Capacidades	OpenAI, Google Gemini, Anthropic, Hugging Face
Desempenho e Eficiência	Groq, Mistral AI, AWS Bedrock
Personalização	Hugging Face, Ollama, Meta LLaMA
Segurança e Privacidade	Azure, AWS, Google Cloud, Cohere
Facilidade de Integração	OpenAI, Hugging Face, AWS Bedrock, GitHub Marketplace
Custo e Precificação	AWS, Google Cloud, Hugging Face, OpenAI
Suporte Multilíngue	Google Gemini, DeepSeek, Cohere, Hugging Face
Compliance e Regulamentação	Azure, AWS, IBM, Google Cloud

- **Cuidado**: Tudo está mudando muito rapidamente neste mercado.

Frameworks que facilitam a interface com diversas APIs

- **Situação típica**: Escolhi o fornecedor escolhi o modelo, e agora?
  - Você pode acessar direto a API usando requisições HTTRs.
  - Vantagem: Controle total do que você está fazendo.
  - Desvantagens:
    - Chamada especifica para cada API.
    - Código menos portável e mais trabalhoso para migrar entre fornecedores.
    - Mais lento de forma geral! Você terá que fazer tudo 'sozinho'.
- Existem diversos *frameworks* facilitadores.
  - Já integram diversas APIs de forma unificada.
  - Aumenta a produtividade (menos código para você fazer).
  - Soluções mais robustas (não fique dependente de apenas um fornecedor).
- Costumo fazer uma distinção entre os *frameworks*:
  - *Frameworks* para acessar modelos de linguagem.
  - *Frameworks* para desenvolver aplicações baseadas em modelos de linguagem.

Frameworks para acessar Modelos de Linguagem

- **Objetivo principal**: Facilitam o uso de um LLM como uma ferramenta dentro da linguagem de programação.
- **Interface unificada**: Mesma interface de conexão com diversos fornecedores.
- **Facilidade de prototipagem**: Ajuda a prototipar mais rápido por abstrair tarefas comuns:
  - Interface de chat com streaming.
  - Chamada de ferramentas.
  - Saída estruturada.
- **Troca fácil de provedor**: Alterar o fornecedor de LLM é feito com apenas uma linha de código.
- **Foco na experiência do cientista**: O LLM passa ser mais uma ferramenta dentro do *toolkit*.

Frameworks que já usei e recomendo

- chatlas · Python.
- llm · Python.
- ellmer · R.
- tidychatmodels · R.

- **Não é uma lista exaustiva!**

- ellmer e chatlas são desenvolvidos pela https://posit.co/.

- Bibliotecas muito similares.
- Aprende um e leve outro :)

Frameworks para desenvolver aplicações

- **Objetivo principal**: Facilitar o desenvolvimento de aplicações inteligentes baseadas em modelos de linguagem (LLMs).
- **Interação com múltiplas fontes de dados**: Integração com APIs, bancos de dados, documentos e outros sistemas externos.
- **Facilidade de orquestração de fluxos de trabalho**: Permitem a criação de fluxos complexos de informação entre diferentes componentes.
- **Customização e escalabilidade**: Suporte para personalização de modelos e construção de aplicações escaláveis e dinâmicas.
- **Retrieval Augmented Generation (RAG)** e **IA Agents**.

Frameworks populares

- LangChain.
- LangGraph.
- LlamaIndex.
- AutoGen.
- PydanticAI.
- Google Trends
- **Trabalhar o Script 2**.

Projetando aplicações de LLMs

O que é uma aplicação de LLMs?

- Uma aplicação de LLM deve resolver um problema do usuário.
- Etapas:
  - Domínio do problema do usuário.
  - Domínio do modelo.
  - Converter o problema do usuário para o domínio do modelo.
  - Converter a saída do modelo para o domínio do usuário.
- Layer de transformação entre dois 'mundos'.
- Tipos de modelos: *chat* versus *completion*.
- Atualmente trabalhamos direto com *chat models*.

Anatomia do Loop

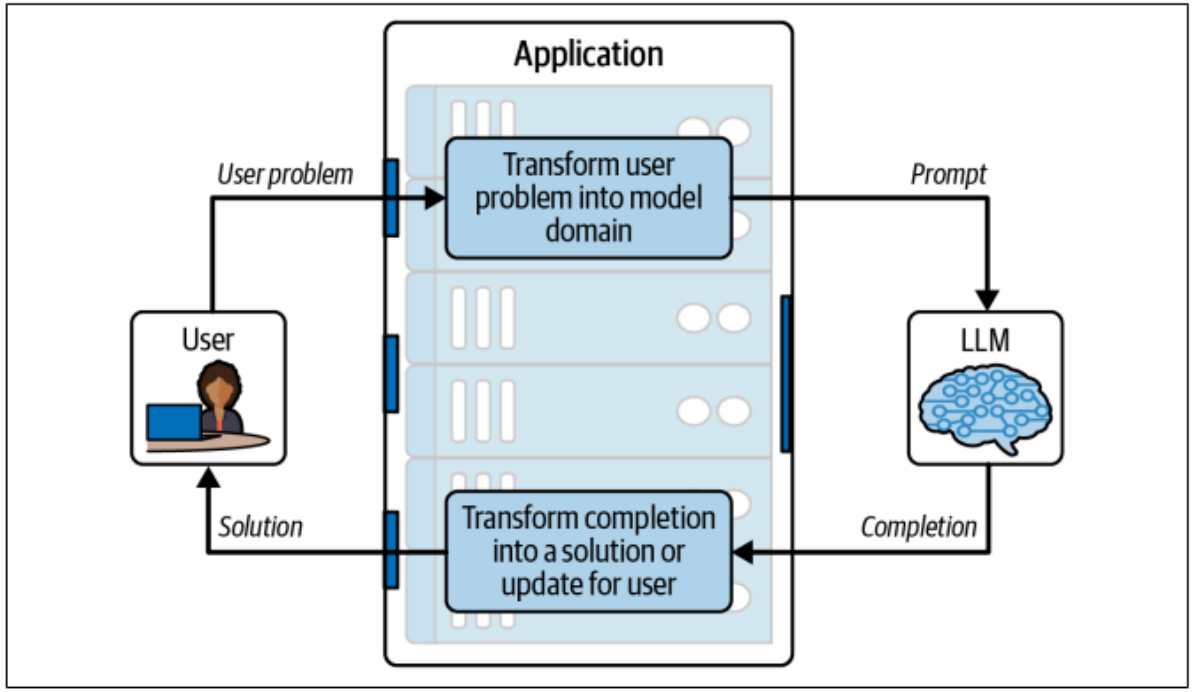


Figure 4-1. LLM-based applications implement the loop, which conveys information from the user domain to the LLM's text domain and then back

Fonte: Prompt Engineering for LLMs. Pag. 66.

O problema do usuário



Exemplos - Complexidade crescente

Dimensão	Revisão de Texto	Suporte de TI	Analista de Dados
Meio	Texto	Voz por telefone	Dashboards interativos, consultas em linguagem natural e integração com bancos de dados
Nível de Abstração	Problema concreto, bem definido e pequeno	Espaço de problema amplo e abstrato, com espaço de solução grande, porém limitado pela documentação disponível	Envolve compreender as métricas de negócio relevantes e traduzi-las em análises e visualizações adequadas
Contexto Necessário	Apenas o texto enviado pelo usuário	Acesso pesquisável à documentação técnica e transcrições de atendimentos anteriores	Acesso a schemas de banco de dados, dicionários de dados, histórico de relatórios e documentação de KPIs
Statefulness	Sem estado — cada chamada à API contém um problema independente	Deve rastrear o histórico da conversa e as soluções já tentadas	Deve rastrear o contexto das análises anteriores, filtros aplicados e decisões tomadas ao longo da sessão

Convertendo para o domínio do modelo

Para que o problema seja resolvido o prompt deve:

- Parecer com documentos do conjunto de treinamento.
- **Incluir todas as informações relevantes para resolver o problema.**
- Deve condicionar o modelo a gerar uma resposta útil.
- Ter um ponto de parada claro e natural.

Princípio Little Red Riding Hood

- Quanto mais o prompt se parecer com os documento do conjunto de treino, mais previsível e estável será a *completion*.
- Usar padrões familiares:
  - Markdown.
  - Código.
  - Q&A.
  - Article e homework.

Exemplo de prompt

```
# Data Analysis and Statistical Modeling 101 - Homework Assignment

Provide answers for the following problems. Each answer should be concise,
clear, and based on sound statistical reasoning.

## Problem 1
A dataset contains customer ages with mean = 35 and standard deviation = 10.
What can you say about the distribution of ages? Provide a brief interpretation.

## Solution 1
The average customer age is 35, with moderate variability, meaning most customers
are likely between 25 and 45 years old assuming an approximately normal distribution.

## Problem 2
You are analyzing a dataset of an e-commerce platform. The conversion rate dropped
from 3.2% to 2.1% after a website redesign.

Additional context:
- Sample size before redesign: 50,000 users
- Sample size after redesign: 52,000 users
- No major marketing changes occurred during the period

Please provide:
1. A statistical interpretation of the change
2. Whether this difference is likely significant
3. A recommendation for next steps

## Solution 2
```

Analisando os componentes do prompt

- Cabeçalho
  - Imita um tipo de documento comum.
  - Contexto acadêmico.
  - Tom técnico.
  - Expectativa de resposta estruturada.
  - Modelo em modo 'analítico'.
- Instruções Globais
  - Define o estilo da resposta.
  - Critério de qualidade.
  - Controle de comportamento via contexto.
- Exemplo demonstrativo
  - Few-shot prompting.
  - Ensina implicitamente: Formato, estilo e conteúdo.
  - Modelo aprende por imitação.
- Natureza do exemplo
  - Problema simples.
  - Solução direta e sem ambiguidade.
  - Âncora cognitiva.
- Estrutura do problema real
  - Contexto aplicado para um cenário 'realista'.
  - Adiciona contexto explícito.
- Estrutura da tarefa
  - Interpretação.
  - Significância.
  - Recomendação.

4 Etapas da tradução



Além do Loop Simples



Estado Persistente

Aplicações complexas mantêm histórico entre chamadas. Chat sessions exigem registro de toda a conversa — e resumos quando o contexto estoura.



Contexto Externo (RAG)

LLMs não têm todas as respostas. RAG augmenta o prompt com dados privados, recentes ou sigilosos via busca vetorial ou índices tradicionais.



Raciocínio Mais Profundo

Chain-of-Thought força o modelo a 'pensar em voz alta' antes de responder. Cada token gerado influencia os próximos — o pensamento ocorre na completion.



Uso de Ferramentas (Tools)

O modelo sozinho não age no mundo. Com tools, ele gera chamadas de função que a aplicação executa em APIs reais — lendo e escrevendo no mundo externo.

Resumo geral dos componentes da aplicação

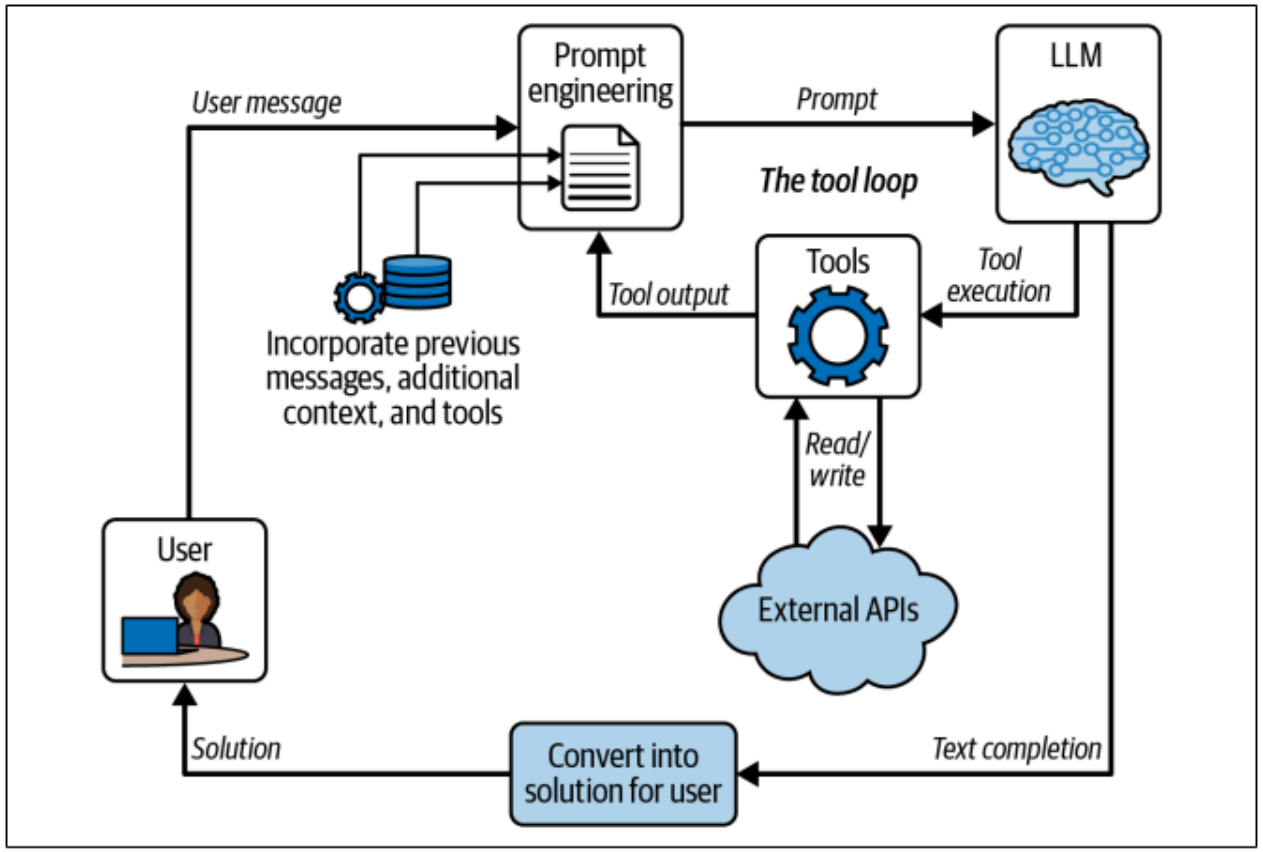


Figure 4-3. A more complicated application loop that includes an internal tool loop

Fonte: Prompt Engineering for LLMs. Pag.80

Avaliando aplicações de LLMs

Avaliando aplicações de LLMs

Por que avaliar?

- LLMs são probabilísticos.
- Pequenas mudanças -> grandes variações de comportamento.
- Qualidade não é óbiva -> precisa ser medida continuamente.
- Avaliação é parte do ciclo de desenvolvimento.

Tipos de avaliação

- Avaliação Offline (antes do deploy)
  - Testa hipóteses sem impactar usuários.
  - Usa dados simulados ou históricos.
  - Permite comparar versões (A vs B).
- Avaliação Online (em produção)
  - Mede comportamento real dos usuários.
  - Baseada em telemetria + feedback.
  - Detecta degradação ao longo do tempo.

Avaliação Offline

Objetivo: Validar qualidade antes de expor ao usuário

1. Testes baseados em dataset
   - Conjunto de perguntas/respostas esperadas.
   - Avaliação automática ou semi-automática.
2. LLM como juiz (LLM-as-a-judge)
   - Modelo avalia respostas de outro modelo.
   - Critérios: Relevância, correção, clareza, completude.
3. Testes funcionais (quando possível)
   - Código roda?
   - API responde corretamente?
   - Output respeita formato esperado?
   - ...

KPIs Offline

- Acurácia/correção.
- Escore de relevância.
- Taxa de alucinação.
- *Format compliance (%)*.
- Consistência (mesma pergunta -> mesma resposta).
- Latência (tempo de resposta).

Avaliação Online

Objetivo: Mede valor real para o usuário

1. Engajamento
   - Taxa de interação (mensagens por sessão).
   - Retenção de usuários.
   - Tempo de sessão (tem que ver se faz sentido).
2. Qualidade percebida
   - 👍 / 👎 (quando disponível).
   - Taxa de resposta aceita.
   - Taxa de reescrita manual.
3. Eficiência
   - Tempo até resolução do problema.
   - Número de interações até resposta final.
   - Taxa de sucesso da tarefa.
1. Confiabilidade
   - Taxa de fallback ("não sei responder").
   - Taxa de erro (respostas inválidas).
   - Taxa de uso de fontes corretas (em RAG).

Não tem escolha certa! Tudo é aplicação dependente :(

- Avaliar LLMs ≠ Avaliar modelos tradicionais.
- Precisamos avaliar utilidade + confiabilidade + consistência.
- Sistema nunca está pronto! Precisa sempre ser monitorado e adaptado.

Prompt Engineering

O que é Prompt Engineering?

Definição:

Prompt Engineering é o processo de **criar e refinar comandos (prompts)** para se comunicar de forma eficaz com modelos de inteligência artificial (IA), especialmente modelos de linguagem como o GPT-3 ou GPT-4.

- Envolve **clareza, especificidade e contexto**.
- Foco em **respostas relevantes e de alta qualidade**.

Por que é importante?

- Influência diretamente a **qualidade das respostas** da IA.
- Melhora a compreensão de **contexto, intenção e nuances**.
- Gera uma **experiência de usuário mais eficaz e produtiva**.

Principais benefícios

- Respostas mais **precisas e coerentes**.
- Conteúdo mais **alinhado com as expectativas**.
- Reduz o número de tentativas para obter uma boa resposta.
- **Economiza tempo e recursos**.
- Ideal para uso em ambientes **profissionais e automatizados**.

Considerações gerais

- Leitura recomendada Getting started with AI: Good enough prompting.
- Ideia de *prompt* **bom o suficiente** é essencial.
- Analogia usada no artigo (traduzida para o português):

"Trate a IA como um novo colega de trabalho infinitamente paciente, que esquece tudo o que você diz a cada nova conversa, alguém altamente recomendado, mas cujas habilidades reais não são tão claras. Duas partes disso são análogas ao trabalho com humanos (ser novo no emprego e ser um colega de trabalho) e duas delas são muito diferentes (esquecer tudo e ser infinitamente paciente). Devemos começar pelo ponto em que as IAs são mais parecidas com os humanos, pois essa é a chave para uma formulação de prompts suficientemente boa."

- Em geral o próprio fornecedor vai oferecer **dicas** de como desenhar o *prompt*.
- Alguns *guidelines*:
  - Anthropic.
  - OpenAI.
  - Gemini.
- Anthropic automatically prompt. Precisa criar uma conta.
- Vou trabalhar principalmente exemplos genéricos nesta etapa.

Componentes do prompt

Conteúdo Estático

Sempre o mesmo — define e clarifica a tarefa geral.

- Instrução geral do problema
- Regras explícitas de formato e tom
- Exemplos (few-shot shots)
- Boilerplate de estrutura do prompt
- **Score: geralmente o mais alto — base da consistência**

Conteúdo Dinâmico

Diferente a cada requisição — detalha usuário e contexto.

- Perfil e preferências do usuário.
- Histórico de interações.
- Dados recuperados via RAG.
- Estado atual da aplicação.
- **Score: variável — quanto mais relevante, maior**

Componentes dinâmicos



Papel de cada componente



Estratégias de prompt

Introdução a engenharia de prompt

Prompt básico

- "Explique o conceito de engenharia de prompt em uma linha."

Prompt estruturado

- "Ofereça uma definição de {topico}, explique a sua importância, e liste os três benefícios principais".

Efeito do prompt

- Diferentes prompts podem levar a respostas muito diferentes sobre o mesmo tópico.
- Teste os seguintes *prompts*:

1. Liste três aplicações sobre inteligência artificial generativa em {seu campo de atuação}.
2. Explique como a inteligência artificial generativa está revolucionando {seu campo de atuação}.
3. Você é um {sua profissão}. Descreva três formas que a inteligência artificial generativa tem melhorado seu dia a dia de trabalho {seu local de trabalho}.

- Quais as principais diferenças?

Papel da engenharia de prompt

Melhorias possíveis

- Estruturação da saída do modelo.
- Melhorar acurácia e relevância da resposta.
- Possibilita resolver tarefas complexas.
- Reduz o viés e melhora a 'honestidade' das respostas.
- Incentiva o modelo a 'pensar' em diferentes formas.
- Combinar diferentes estratégias para resolver uma tarefa.

Exemplos

- Verificação de fatos:

"Avalie a seguinte declaração em termos fatuais. Se incorreto, ofereça a informação correta: {declaracao}."

- Tarefas complexas:

"Resolva o seguinte problema passo-a-passo: Problema: {problema}"

- Problema: Calcule o juros compostos sobre R$1000 investidos por 5 anos a uma taxa anual de 5%, com capitalização anual.

Estruturas básicas de prompt

Visão geral

Estruturas fundamentais

- **Single-turn prompt:** Tentativa única.
- **Multi-turn prompt:** Série de interações com o modelo que mantenha o contexto.
- **Conversation Chains:** Manter o contexto ao longo de múltiplas interações.
- **Prompt Templates:** Estruturas reutilizáveis para prompt consistente.

Exemplos

- Single-turn: O que é GenAI?
- Multi-turn:
  - Olá! Eu estou aprendendo sobre GenAI. Você pode me explicar sobre isso?
  - Quais são as técnicas mais usadas?
  - Como a GenAI está relacionada com ciência de dados?

Prompt templates

Motivação

- **Reuso**: Templates podem ser reutilizados em diferentes contextos.
- **Flexibilidade**: Variáveis permitem dinâmica na inserção de conteúdo.
- **Gerenciar a complexidade**: Lidam com estruturas complexas, incluindo lógica adicional e loops.
- **Escalabilidade**: Se as aplicações crescem, templates tornam mais fácil gerenciar e manter um número grande de prompts.

Exemplos

- "Ofereça uma breve explicação sobre {topico}."
- "Explique o conceito de {conceito} no campo de {campo} para uma audiência {audiencia}, de forma concisa."
- "Meu nome é {nome} e eu tenho {idade} anos. Eu trabalho como {profissão}. Você pode me dar conselhos profissionais baseados nestas informações?"

Outros Templates

Template para categorizar itens

- "Categorize estes items em grupos: {itens}. Forneça as categorias e os itens em cada categoria"
- itens: maça, banana, cenoura, martelo, canivete, alicate, novela, livro, revista.
- "Analize a seguinte lista de items: {itens}. Forneça um resumo da lista e sugira qualquer padrão ou agrupamento".
- itens: Python, JavaScript, HTML, CSS, React, Django, Flask, Node.js, R, Julia, Matlab.

Tarefas com contexto e restrições

- Prompt completo: Por favor, forneça uma solução para a {tarefa}, considerando o {contexto} e que respeite as {restricoes}.
- Exemplo: Por favor, forneça uma solução para {criar um logo para uma tech startup}, considere que {a startup foca em AI-driven soluções}, precisa ser {azul e verde e deve ser simples o suficiente para ser reconhecido quando pequeno}.

Zero-Shot prompting

Zero-Shot prompting

Especificação direta da tarefa

- Exemplo de prompt completo:

"Classifique o sentimento do seguinte texto como positivo, negativo ou neutro. Não explique seu raciocinio, apenas forneça a resposta.

Text: {texto}

Sentiment: "

- Frases para testar:

1. Eu amei o filme! As atuações foram fantásticas.
2. O tempo hoje está como esperado para esta época do ano.
3. Eu estou triste com o serviço que eu recebi deste restaurante.

Especificação de formato

- Exemplo de prompt completo:

"Escreva um artigo curto sobre {topico}. Estruture sua resposta no seguinte formato:

Headline: [Um título atrativo para o artigo] Lead: [Um paragrafo breve introdutório resumindo os principais pontos] Body: [2-3 paragráfos fornecendo mais detalhes] Conclusion: [Uma frase de conclusão ou chamada para ação]"

- Assuntos para testar

1. Generative AI.
2. Estatística aplicada.
3. Teoria da probabilidade.

Multi-step reasoning

- Exemplo de prompt:

"Analise o seguinte texto com relação aos seus principais argumentos, suporte baseado em evidência e potenciais contra argumentos. Forneça sua analise com os seguintes passos:

1. Principal argumento: Identifique e declare a afirmação principal ou tese.
2. Evidências de suporte: Liste os pontos principais ou evidências usadas para baseas o argumento principal ou tese.
3. Contra argumentos: Sugira possíveis objeções ou pontos de vista alternativos ao principal argumento.

Text: {text}

Analysis:"

- Texto para teste: A inteligência artificial generativa vai trazer inúmeros benefícios para a sociedade, como aumento da produtividade. Vai permitir que pessoas que não sabem nada de programação criem softwares de forma profissional. Sendo assim, desenvolvedores e até mesmo cientistas de dados não serão mais necessários.

Análise comparativa (comparative analysis)

Template básico

- Básico: "Explique {topico}".
- Estruturado:

"Explique {topico} por analisar os seguintes pontos:

1. Definição.
2. Fatores chave.
3. Aplicações no mundo real.
4. Potenciais impactos na indústria."

- Tópicos para teste
  - Blockchain.
  - LLMs.

Few-Shot learning

Few-Shot learning

Aprendendo com poucos exemplos

- Tradicional ML precisa de muitos exemplos para aprender uma tarefa.
- Few-Shot e In-Context learning aprendem baseado em poucos exemplos.
- Ideal para tarefas de classificação de texto.

Prompt template

"Classifique o sentimento como Positivo, Negativo ou Neutro.

Exemplos: Text: Eu amo este produto! Ele é fantástico. Sentiment: Positivo

Text: Este filme foi horrível. Eu não gostei dele. Sentiment: Negativo

Text: O tempo hoje é ok. Sentiment: Neutro

Agora, classifique o seguinte

Text: {texto} Sentiment: "

- Frase para testar: "Eu não acredito como esse novo restaurante é bom!"

Multi-task

Estrutura básica

- Definição: Treinar um modelo para realizar múltiplas tarefas relacionadas.
- Benefícios: Melhora a eficiência, melhor generalização, reduz overfitting.

Implementação

- Escrever um prompt que inclui exemplos de múltiplas tarefas.
- Use instruções específicas para guiar o modelo.

Template

"Faça a tarefa específica para o texto fornecido.

Exemplos:

Text: Eu amei este produto! Ele é fantástico. Task: sentimento Result: positivo

Text: Bonjour, comment allez-vous? Task: língua Result: francês

Agora, faça para a seguinte tarefa:

Text: {texto} Task: {tarefa} Result: "

- Frase para teste:

1. Eu não posso acreditar como este restaurante é ótimo.
2. Guten Tag, wie geht es Ihnen?

In-Context Learning

- Permite ao modelo se adaptar a novas tarefas baseado em exemplos.
- Os exemplos são oferecidos diretamente via prompt.
- Aspectos importantes:
  - Não é necessário trabalhar com *fine-tuning*.
  - Flexível: Pode ser adaptado para muitos problemas.
  - Prompt engineering: O prompt é crucial para a performance.
- Tarefa: Forneça a versão feminina para o nome fornecido.
- Exemplos: marcio: marcia, flavio: flavia

Template

" Task: {descricao_tarefa}

Exemplos: {exemplos}

Agora, faça a tarefa na seguinte entrada

Input: {texto_input} Output: "

Boas-práticas e avaliação

Seleção dos exemplos:

- Diversidade: Tente cobrir os diferentes aspectos da tarefa.
- Clareza: Use exemplos não ambíguos.
- Relevância: Escolha exemplos similares ao input esperado.
- Balanço: Assegure um bom balanço de classes/categorias.
- *Edge cases*: Inclua exemplos não usuais e dificeis.

Engenharia de prompt

- Instruções claras: Especifique a tarefa.
- Formato consistente: Mantenha uma estrutura uniforme para os exemplos e inputs.
- Conciso: Evite informação desnecessárias que podem confundir o modelo.

Avaliação

- Crie um diverso conjunto de teste.
- Compare a predição do modelo com os verdadeiros labels.
- Use métricas apropriadas (acurácia, F1 score, etc) de acordo com a tarefa.

Resumo da aula

